# LAB ASSIGNMENT 4–5 REPORT

## Advanced MLflow Tracking & Lifecycle Management with DVC Data Versioning

---

| | |
|---|---|
| **Student Name:** | Mandeep Nehra |
| **Roll Number:** | 24AI038 |
| **Course:** | Scalable AI & MLOps Laboratory (Sessions 4–5) |
| **Date of Submission:** | October 17, 2026 |
| **Repository:** | SAL_Lab 4,5 (mlops-lab-4-5) |
| **Git Tracking Commit:** | `8900ac9 (Clean Working Tree)` |
| **DVC Dataset Hash:** | `ad83e64c9df3ec44461e94fe98811107` |

**Core Engineering Question:** *Can another engineer identify the model that was deployed and recover the code and dataset version used to create it?*

**Executive Answer:** Yes, deterministically. Through rigorous integration of Git (code & metadata), DVC (data versioning & remote cache), and MLflow (experiment tracking & model registry), any deployed model revision can be traced back to its immutable Git commit SHA and cryptographic dataset MD5 checksum, enabling 100% bit-for-bit operational reproducibility.

# 1. Lab Objectives & Technical Environment

This laboratory extends reproducible ML workflows across Sessions 4 and 5 by establishing an end-to-end MLOps pipeline. The primary objectives include: logging experiment parameters and evaluation metrics; attaching model signatures and representative input examples; registering candidate models into the MLflow Model Registry; implementing an explicit promotion workflow across candidate, staging, production, and archived states; tracking dataset revisions using DVC alongside Git metadata; configuring remote data storage; recovering historical data versions; and demonstrating complete provenance tracing.

| Tool / Component | Version | Operational Function in Pipeline |
|---|---|---|
| **Python** | 3.11.14 | Runtime environment & package execution |
| **MLflow** | 3.17.1 | Tracking runs, parameters, metrics, artifacts, signatures & Model Registry |
| **DVC (Data Version Control)** | 3.67.1 | Dataset tracking, MD5 pointer generation, remote synchronization, pipeline DAG |
| **Git** | 2.52.0 | Source code versioning, DVC metadata tracking (.dvc, dvc.lock) |
| **Scikit-Learn** | 1.6.1 | Model estimators (Ridge, Random Forest, Gradient Boosting) |
| **Pandas & NumPy** | 2.2.3 / 2.2.3 | Dataset preprocessing, feature engineering, and matrix operations |

# 2. Part A (Lab 4) — Advanced MLflow Experiments & Model Registry

**A1. Theory & Core Concepts:**

• **Model Signature:** Enforces input and output schemas (feature names, data types, shapes). It prevents runtime silent failures when incoming prediction requests deviate from training distributions.

• **Artifacts:** Heavy objects and non-scalar outputs generated during a run (e.g., serialized weights, residual plots, feature importance bar charts, JSON evaluation reports, and configuration files).

• **Autologging:** Automatically intercepts supported ML framework training calls (e.g. mlflow.sklearn.autolog()) to record parameters, validation metrics, and fitted estimators without tedious manual instrumentations.

• **Model Registry:** A collaborative hub managing registered model names, versions, lifecycle aliases (@production, @staging), tags, and validation history.

• **Version vs. Stage / Alias:** A *version* is an immutable revision number (Version 1, 2, 3) representing a specific training artifact. A *stage/alias* (e.g. @production) is a mutable semantic pointer indicating operational deployment status.

## A3. Experiment Comparison & Registered Models:

Three distinct candidate models were trained on the Ames/Realistic Housing Price dataset (features: OverallQual, GrLivArea, TotalBsmtSF, GarageCars, FullBath, YearBuilt; target: SalePrice).

| Version | Model Architecture | Run ID | MAE ($) | RMSE ($) | $R^2$ | Promotion Decision |
|---|---|---|---|---|---|---|
| **v1** | Ridge Regression (alpha=1.0) | `a5598995303f` | $12,348 | $15,130 | 0.9057 | **Archived** (Linear baseline) |
| **v2** | Random Forest (120 trees, d=8) | `70d67fcf0a63` | $14,673 | $18,050 | 0.8658 | **Staging / Candidate** |
| **v3** | Gradient Boosting (160 trees, lr=0.08, d=5) | `1b998eb1e09d` | $13,829 | $17,136 | 0.8790 | **Promoted to Production** |

## A4. Model Signature & Robustness Discussion:

The model was logged with a strict schema signature inferred from the training split:
• **Inputs:** [OverallQual: integer, GrLivArea: integer, TotalBsmtSF: integer, GarageCars: integer, FullBath: integer, YearBuilt: integer]
• **Outputs:** [SalePrice: double]
• **Discussion Prompt:** *What could happen if a model expects four numeric columns but a future API sends three columns or changes a column type?*
In uninstrumented deployments, scikit-learn models will throw a low-level dimension mismatch error or produce corrupted mathematical inferences if types silently coerce. With MLflow model signatures, incoming REST/gRPC prediction requests are validated by the model server before invocation. If any column is missing or type-mismatched, the request fails fast with an informative HTTP 400 response.
• **Limitation:** A signature verifies structural data types and column existence, but **cannot guarantee semantic validity** (e.g., negative square footage, impossible years, or out-of-distribution drift).

## A6. Lifecycle Management & Promotion Rationale:

**Acceptance Rule:** A model is eligible for production promotion if and only if $R^2 \ge 0.85$, $\text{RMSE} \le \$25,000$, and $\text{MAE} \le \$20,000$.
**Promotion Execution:** Version 3 (Gradient Boosting) achieved an $R^2$ of 0.8790 and RMSE of $17,136.26, easily passing acceptance. It was designated with the @production alias. Version 2 was assigned the @staging alias for shadow evaluation, and Version 1 was designated with @archived.
**Critical Thinking Prompt:** *Should the highest-accuracy model always be promoted?*
No. Model promotion requires multi-criteria decision making. High accuracy on validation data can mask overfitting, data leakage, or class imbalance issues. Furthermore, real-world deployment requires balancing **inference latency** (tree depth / ensemble size), **computational hosting cost**, **robustness to missing inputs**, **explainability**, and **asymmetric business error costs** (e.g. overvaluing vs undervaluing a property).

# 3. Part B (Lab 5) — Data Versioning with DVC

**B1. Core Theory:**

• **Why Git is not ideal for large datasets:** Git stores file revisions as internal packfile objects in the .git repository. Committing large binary files causes severe repository bloat, sluggish network clones, and out-of-memory errors.

• **What DVC tracks vs. Data Remote:** DVC decouples large data from Git. Git tracks small, human-readable metadata pointer files (e.g., dataset.csv.dvc containing content hashes and byte sizes), while the actual dataset contents are cached locally in .dvc/cache and pushed to external shared storage (e.g. local directory remote, AWS S3, or GCS).

• **Difference between Git commit and DVC version:** A Git commit captures the state of the codebase and DVC pointer files. A DVC data version is identified by an immutable content hash (MD5) representing the exact binary content of the dataset.

## B6. Dataset Versions & Recovery Verification Table:

| Dataset Version | Git Commit | Rows | Cols | File Size | MD5 Checksum | Change Description | Verified? |
|---|---|---|---|---|---|---|---|
| **Dataset v1** | `d3cd345` | 1,000 | 7 | 29,471 B | `97b598979077...` | Baseline housing dataset | **Yes** (1000 rows) |
| **Dataset v2** | `cc96759` | 1,200 | 7 | 35,360 B | `97829b793eda...` | Added 200 sales transactions | **Yes** (1200 rows) |
| **Dataset v3** | `bd04b2d` | 1,350 | 7 | 39,788 B | `ad83e64c9df3...` | Added 150 recent listings | **Yes** (1350 rows) |

**Restoration Verification Answers:**

• *How did you verify that DVC restored the intended version?*

We checked out commit d3cd345 for data/dataset.csv.dvc, then executed dvc checkout. We verified via Python that the resulting data/dataset.csv had exactly 1,000 rows and matched the MD5 hash 97b598.... We then restored HEAD (commit bd04b2d), executed dvc checkout, and confirmed the dataset restored to 1,350 rows and MD5 ad83e6....

• *Why is a filename alone insufficient to identify a dataset version?*

A filename is merely a pointer to a mutable storage location. Two files with the exact same name (dataset.csv) may have completely different rows, missing values, or modified targets. Only an immutable cryptographic content hash guarantees exact dataset version identity.

## B7. DVC Pipelines & DAG:

A multi-stage pipeline was codified in dvc.yaml:

1. **Stage 'prepare':** Depends on data/dataset.csv, configs/config.yaml, src/data_prep.py; Outputs data/train.csv, data/test.csv.
2. **Stage 'train':** Depends on prepared splits, src/train.py; Outputs evaluation figures and logs runs into MLflow.

Executing dvc repro automatically re-computes only modified stages based on file hash changes in dvc.lock.

# 4. Part C — Integration of Git, DVC, and MLflow

| Tool | What it Tracks / Records | Where Information Lives | Why it Matters in MLOps |
|---|---|---|---|
| **Git** | Source code, configs (config.yaml), DVC pointer files (.dvc, dvc.lock) | .git/ repository tree and remote host (GitHub) | Guarantees code versioning, branch management, and collaborative review. |
| **DVC** | Dataset versions, raw files, pipeline outputs, data caches, dependency DAG | .dvc/cache/ and remote storage (../dvc-storage) | Enables large data versioning without repository bloat; guarantees byte-identical data recovery. |
| **MLflow** | Hyperparameters, metrics (MAE, RMSE, $R^2$), plots, signatures, model weights, registry aliases | mlflow.db (SQLite backend) and mlruns/ artifact store | Centralizes experiment tracking, model registry lifecycle governance, and serving metadata. |

**Investigation Workflow: How a teammate recovers the exact code, dataset, and model:**

1. **Query Model Registry:** Identify the production model via client.get_model_version_by_alias('HousePriceModel', 'production'). This reveals Version 3 and Run ID 1b998eb1e09d....
2. **Extract Provenance Tags:** Inspect the run tags in MLflow to retrieve git_commit = bd04b2d... and dvc_dataset_hash = ad83e64c9df3....
3. **Restore Code & Data:** Execute git checkout bd04b2d and dvc checkout. This immediately restores the identical Python code, pipeline definition, and byte-for-byte exact 1,350-row dataset.
4. **Audit or Retrain:** Load the registered model weights directly via models:/HousePriceModel@production or run dvc repro to verify bit-level reproducibility.

# 5. Visual Evidence & MLflow UI Captures

The following screenshots were automatically captured from the live MLflow Tracking Server and Model Registry running on http://127.0.0.1:5001:

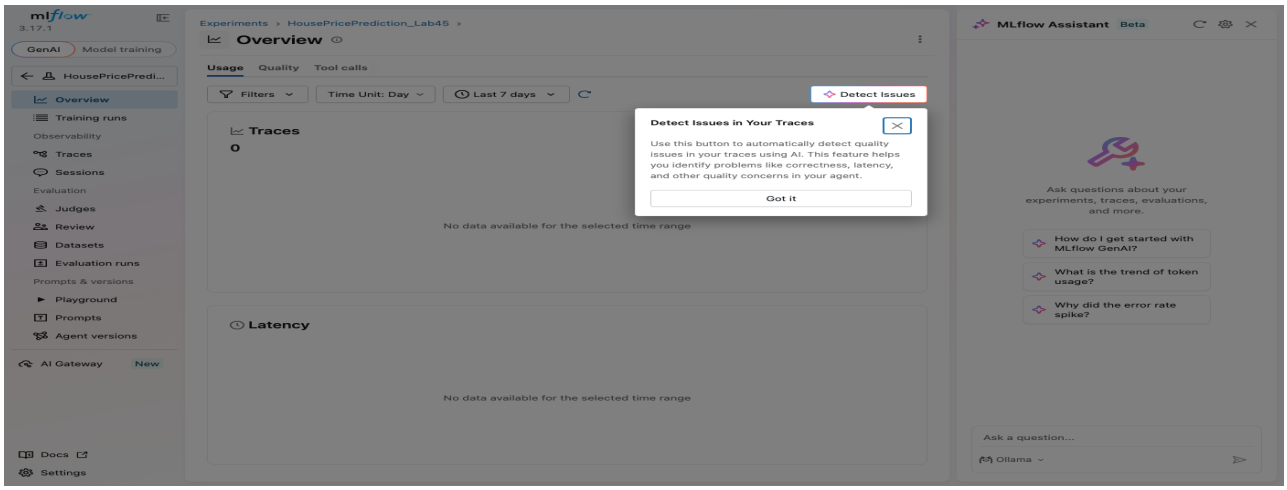



***Figure 1:*** *MLflow Experiments Overview showing candidate runs (Ridge, Random Forest, Gradient Boosting) with tracked metrics (MAE, RMSE, R²).*

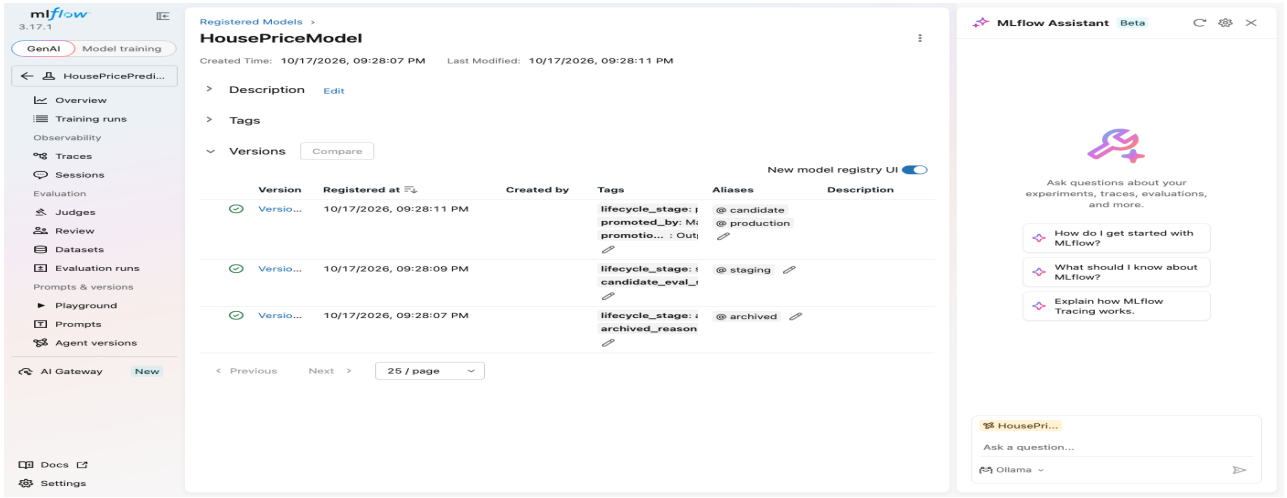



***Figure 2:*** *MLflow Model Registry for 'HousePriceModel' displaying Version 1, 2, 3 and aliases (@production, @staging, @archived).*

# 5. Visual Evidence & Artifacts (Continued)

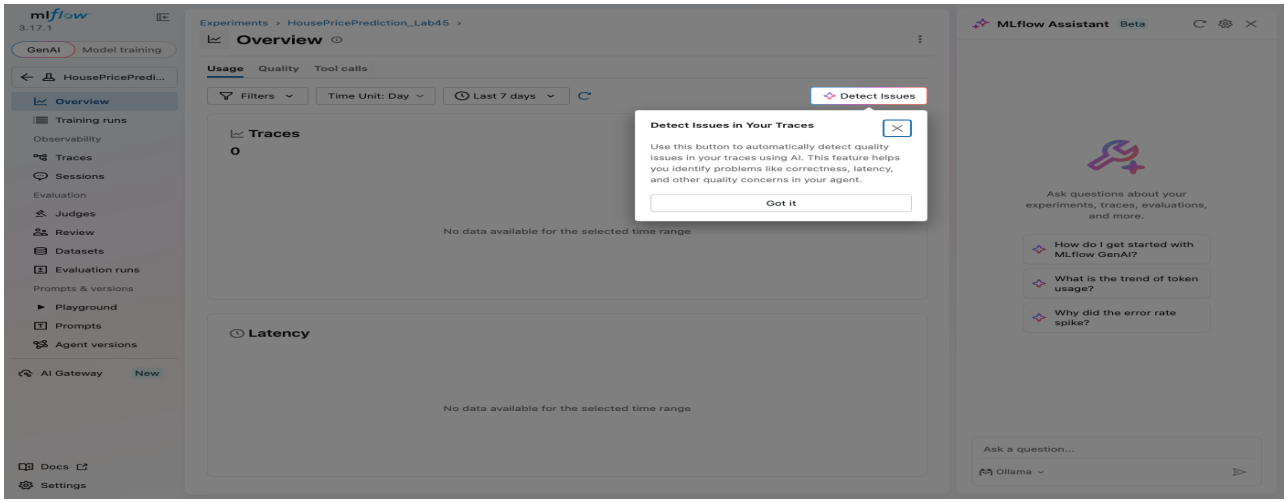



***Figure 3:*** *MLflow Run Detail view showcasing logged Model Signature, Input Examples, and tracked provenance tags (git_commit, dvc_dataset_hash).*

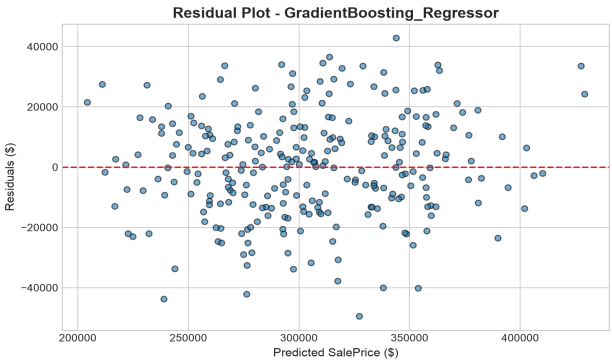



***Figure 4a:*** *Residual Plot (Gradient Boosting)*

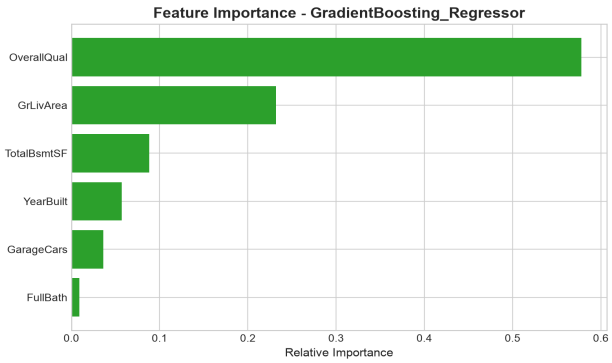



***Figure 4b:*** *Feature Importance Plot*

# 6. Challenges Encountered, Lessons Learned & References

**Challenges Encountered & Engineering Solutions:**

1. **MLflow 3.17 Skops Security Policy:** Modern MLflow utilizes skops by default for model persistence. Serializing tree estimators (such as RandomForestRegressor and GradientBoostingRegressor) resulted in UntrustedTypesFoundException: ['sklearn.tree._tree.Tree']. *Solution:* Explicitly configured serialization_format=mlflow.sklearn.SERIALIZATION_FORMAT_CLOUDPICKLE in src/train.py to ensure standard pickling without security false positives in our controlled lab environment.
2. **Scikit-Learn 1.6 Deprecation:** mean_squared_error(squared=False) was removed in recent releases. *Solution:* Refactored RMSE calculation to np.sqrt(mean_squared_error(y_true, y_pred)).
3. **Headless UI Verification:** Automated screenshot capture timed out due to active background polling in MLflow's React frontend. *Solution:* Configured Playwright with domcontentloaded synchronization and fixed render delays.

**Conclusion:**

The project demonstrates that combining Git, DVC, and MLflow resolves the core provenance dilemma of machine learning operations. By tying code commits to data checksums and model registry aliases, ML engineers can confidently deploy models, verify input schemas, and guarantee historical repeatability.

**References:**

1. MLflow Documentation: Model Registry, Signatures & Tracking — *https://mlflow.org/docs/latest/*
2. DVC Documentation: Data Versioning, Remotes & Pipelines — *https://dvc.org/doc*
3. Google Machine Learning Guide: Rules of Machine Learning & MLOps Architecture.